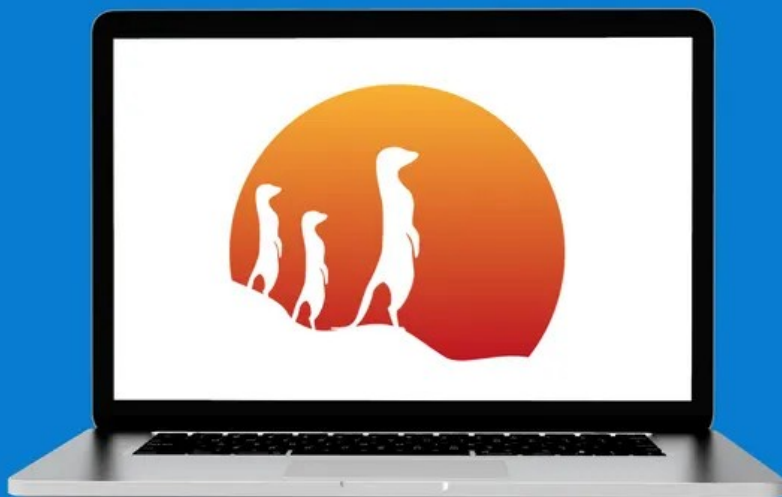


Configuración IDS/IPS Suricata



Paula Fernández

Sumario

Instalación Suricata.....	3
Configurar Suricata para monitorizar la red.....	4
Reiniciar y verificar el servicio.....	4
Lanzar Suricata en modo testeo.....	5
Crear una regla para detectar ICMP (Ping).....	5
Comprobación de alertas.....	6
Modificar la variable HOME_NET.....	6
Repetir comprobación final.....	7

Instalación Suricata.

Creamos una red NAT para hacer esta tarea:



Y ahora añadimos los repositorios de suricata y lo instalamos.

```

usuario@bash: ~
$ sudo add-apt-repository ppa:oisf/suricata-stable
Repository: deb https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu/ jammy main
Description:
Suricata IDS/IPS/NSM stable packages
https://suricata.io/
https://oisf.net/

Suricata IDS/IPS/NSM - Suricata is a high performance Intrusion Detection and Prevention System and Network Security Monitoring engine.
Open Source and owned by a community run non-profit foundation, the Open Information Security Foundation (OISF). Suricata is developed by the OISF, its supporting vendors and the community.
This Engine supports:
- Multi-Threading - provides for extremely fast and flexible operation on multicore systems.
- Multi Tenancy - Per Vlan/Per Interface
- Uses Rust for most protocol detection/parsing
- TLS/SSL certificate matching/logging
- JA3 TLS client fingerprinting
- JA3S TLS server fingerprinting
- IEEE 802.1ad (QinQ) and IEEE 802.1Q (VLAN) support
- VXLAN support
- All JSON output/logging capability
- IDS runmode
- IPS runmode
- IDPS runmode
- NSM runmode
- eBPF/XDP
- Automatic Protocol Detection and logging - IPV4/6, TCP, UDP, ICMP, HTTP, SMTP, TLS, SSH, FTP, SMB, DNS, NFS, TFTP, KRBS, DHCP, IKEv2, SNMP, SIP, RDP
- SCADA automatic protocol detection - ENIP/DNP3/MODBUS
- File Extraction HTTP/SMTP/FTP/NFS/SMB - over 4000 file types recognized and extracted from live traffic.
- File MD5/SHA1/SHA256 matching
- Gzip Decompression
- Fast IP Matching
- Datasets matching
- Rustlang enabled protocol detection
- Lua scripting
and many more great features -
https://suricata.io/features/all-features/
Más información: https://launchpad.net/~oisf/+archive/ubuntu/suricata-stable
Añadiendo repositorio.
Oprima [INTRO] para continuar o Ctrl+c para cancelar.
Adding deb entry to /etc/apt/sources.list.d/oisf-ubuntu-suricata-stable-jammy.list
Adding disabled deb-src entry to /etc/apt/sources.list.d/oisf-ubuntu-suricata-stable-jammy.list
Adding key to /etc/apt/trusted.gpg.d/oisf-ubuntu-suricata-stable.gpg with fingerprint 121504ADE276E141A0704A75AC10378CF205C960

usuario@bash: ~
$ sudo apt-get install suricata
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
libevent-core-2.1-7 libevent-pthreads-2.1-7 libhiredis0.14 libhyperscan5 liblua5.1-common libnet1 libnetfilter-queue1
Se instalarán los siguientes paquetes NUEVOS:
libevent-core-2.1-7 libevent-pthreads-2.1-7 libhiredis0.14 libhyperscan5 liblua5.1-common libnet1 libnetfilter-queue1 suricata
0 actualizados, 8 nuevos se instalarán, 0 para eliminar y 336 no actualizados.
Se necesita descargar 7.206 kB de archivos.
Se utilizarán 33,7 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]
Des1 http://es.archive.ubuntu.com/ubuntu jammy/universe amd64 libhyperscan5 amd64 5.4.0-2 [2,485 kB]
Des2 https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu jammy/main amd64 suricata amd64 1:8.0.3-0ubuntu1 [4,481 kB]
Des3 http://es.archive.ubuntu.com/ubuntu jammy/main amd64 libevent-core-2.1-7 amd64 2.1.12-stable-1build3 [93,9 kB]
Des4 http://es.archive.ubuntu.com/ubuntu jammy/main amd64 libevent-pthreads-2.1-7 amd64 2.1.12-stable-1build3 [7,042 B]
Des5 http://es.archive.ubuntu.com/ubuntu jammy/universe amd64 libhiredis0.14 amd64 0.14.1-2 [32,0 kB]
Des6 http://es.archive.ubuntu.com/ubuntu jammy/universe amd64 liblua5.1-common all 2.1.0-beta3+dfsg-6 [44,3 kB]
Des7 http://es.archive.ubuntu.com/ubuntu jammy/main amd64 libnet1 amd64 1.1.6+dfsg-3.1build3 [46,9 kB]
Des8 http://es.archive.ubuntu.com/ubuntu jammy/universe amd64 libnetfilter-queue1 amd64 1.0.5-2 [14,4 kB]
Descargados 7.206 kB en 5s (1.474 kB/s)
Preconfigurando paquetes ...
Seleccionando el paquete libhyperscan5 previamente no seleccionado.
(Leyendo la base de datos ... 206387 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../0-libhyperscan5_5.4.0-2_amd64.deb ...
Desempaquetando libhyperscan5 (5.4.0-2) ...
Seleccionando el paquete libevent-core-2.1-7:amd64 previamente no seleccionado.
Preparando para desempaquetar .../1-libevent-core-2.1-7_2.1.12-stable-1build3_amd64.deb ...
Desempaquetando libevent-core-2.1-7:amd64 (2.1.12-stable-1build3) ...
Seleccionando el paquete libevent-pthreads-2.1-7:amd64 previamente no seleccionado.
Preparando para desempaquetar .../2-libevent-pthreads-2.1-7_2.1.12-stable-1build3_amd64.deb ...
Desempaquetando libevent-pthreads-2.1-7:amd64 (2.1.12-stable-1build3) ...
Seleccionando el paquete libhiredis0.14:amd64 previamente no seleccionado.
Preparando para desempaquetar .../3-libhiredis0.14_0.14.1-2_amd64.deb ...
Desempaquetando libhiredis0.14:amd64 (0.14.1-2) ...
Seleccionando el paquete liblua5.1-common previamente no seleccionado.
Preparando para desempaquetar .../4-liblua5.1-common_2.1.0-beta3+dfsg-6_all.deb ...
Desempaquetando liblua5.1-common (2.1.0-beta3+dfsg-6) ...
Seleccionando el paquete libnet1:amd64 previamente no seleccionado.
Preparando para desempaquetar .../5-libnet1_1.1.6+dfsg-3.1build3_amd64.deb ...
Desempaquetando libnet1:amd64 (1.1.6+dfsg-3.1build3) ...
Seleccionando el paquete libnetfilter-queue1:amd64 previamente no seleccionado.
Preparando para desempaquetar .../6-libnetfilter-queue1_1.0.5-2_amd64.deb ...
Desempaquetando libnetfilter-queue1:amd64 (1.0.5-2) ...
Seleccionando el paquete suricata previamente no seleccionado.

```

Configurar Suricata para monitorizar la red.

Antes de configurar nada, necesitamos saber el nombre de la interfaz de red que vamos a monitorizar, lo haremos con `ip addr`.

```

usuario@bash: ~
usuario@bash:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a3:e2:66 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.21/24 brd 192.168.0.255 scope global noprefixroute enp0s3
        valid_lft forever preferred_lft forever
    inet6 fe80::7edb:72ff:b898:2f16/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

Editamos el fichero de configuración principal para asegurar que Suricata escucha en la interfaz correcta y buscamos la sección `af-packet` para ver si el nombre de la interfaz es el mismo.

```

GNU nano 6.2 /etc/suricata/suricata.yaml
# Linux high speed capture support
af-packet:
- interface: enp0s3
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
  # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
  # This is only supported for Linux kernel > 3.1
  # possible value are:
  # * cluster_flow: all packets of a given flow are sent to the same socket
  # * cluster_cpu: all packets treated in kernel by a CPU are sent to the same socket
  # * cluster_qm: all packets linked by network card to a RSS queue are sent to the same
  # socket. Requires at least Linux 3.14.
  # * cluster_ebpf: eBPF file load balancing. See doc/userguide/capture-hardware/ebpf-xdp.rst for
  # more info.
  # Recommended modes are cluster_flow on most boxes and cluster_cpu or cluster_qm on system
  # with capture card using RSS (requires cpu affinity tuning and system IRQ tuning)
  # cluster_rollover has been deprecated; if used, it'll be replaced with cluster_flow.
  cluster-type: cluster_flow
  # In some fragmentation cases, the hash can not be computed. If "defrag" is set
  # to yes, the kernel will do the needed defragmentation before sending the packets.
  defrag: yes

```

Reiniciar y verificar el servicio.

```

usuario@bash:~$ sudo systemctl restart suricata
usuario@bash:~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IPS/NSM/FW daemon
   Loaded: loaded (/lib/systemd/system/suricata.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-01-22 00:22:14 CET; 8s ago
     Docs: man:suricata(8)
           man:suricata-sc(8)
           https://suricata.io/documentation/
  Process: 6322 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, status=0/SUCCESS)
 Main PID: 6323 (Suricata-Main)
    Tasks: 10 (limit: 4607)
   Memory: 40.6M
      CPU: 329ms
   CGroup: /system.slice/suricata.service
           └─6323 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid --user suricata --group suricata

ene 22 00:22:14 bash systemd[1]: Starting Suricata IDS/IPS/NSM/FW daemon...
ene 22 00:22:14 bash systemd[1]: Started Suricata IDS/IPS/NSM/FW daemon.
ene 22 00:22:14 bash suricata[6323]: i: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode
ene 22 00:22:14 bash suricata[6323]: W: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
ene 22 00:22:14 bash suricata[6323]: W: detect: 1 rule files specified, but no rules were loaded!
ene 22 00:22:14 bash suricata[6323]: i: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
ene 22 00:22:14 bash suricata[6323]: i: threads: Threads created -> W: 4 FM: 1 FR: 1 Engine started.

```

Lanzar Suricata en modo testeo.

Lanzamos Suricata en modo testeo para verificar que no haya errores de sintaxis en el fichero `suricata.yaml`

```
usuario@bash: ~  
usuario@bash:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v  
Notice: suricata: This is Suricata version 8.0.3 RELEASE running in SYSTEM mode  
Info: cpu: CPUs/cores online: 4  
Info: suricata: Running suricata under test mode  
Info: suricata: Setting engine mode to IDS mode by default  
Info: exception-policy: master exception-policy set to: auto  
Info: suricata: Preparing unexpected signal handling  
Info: logopenfile: fast output device (regular) initialized: fast.log  
Info: logopenfile: eve-log output device (regular) initialized: eve.json  
Info: logopenfile: stats output device (regular) initialized: stats.log
```

Crear una regla para detectar ICMP (Ping)

Ahora tenemos que crear el directorio y el archivo vacío, en este caso sería `/etc/suricata/rules/local.rules` y la regla sería esta:

```
usuario@bash: ~  
GNU nano 6.2 /etc/suricata/rules/local.rules *  
alert icmp any any -> any any (msg:"ALERTA: Paquete ICMP detectado"; sid:1000001; rev:1;)S
```

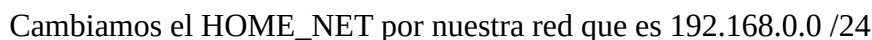
Ahora tenemos que configurar el archivo `suricata.yaml` para que lea el archivo que hemos creado y buscamos la sección `rule-files`.

```
usuario@bash: ~  
GNU nano 6.2 /etc/suricata/suricata.yaml *  
  
# When auto-config is enabled the hashmode specifies the algorithm for  
# determining to which stream a given packet is to be delivered.  
# This can be any valid Napatech NTPL hashmode command.  
#  
# The most common hashmode commands are: hash2tuple, hash2tuplesorted,  
# hash5tuple, hash5tuplesorted and roundrobin.  
#  
# See Napatech NTPL documentation other hashmodes and details on their use.  
#  
# This parameter has no effect if auto-config is disabled.  
#  
hashmode: hash5tuplesorted  
  
##  
## Configure Suricata to load Suricata-Update managed rules.  
##  
default-rule-path: /var/lib/suricata/rules  
  
rule-files:  
- /etc/suricata/rules/local.rules
```

Tras guardas, reiniciamos el servicio con `sudo systemctl restart suricata`. Y desde una maquina con la misma red hacemos un ping a nuestra maquina con suricata. Mientras tanto, en nuestra maquina con suricata hacemos un `sudo tail -f /var/log/suricata/fast.log`.

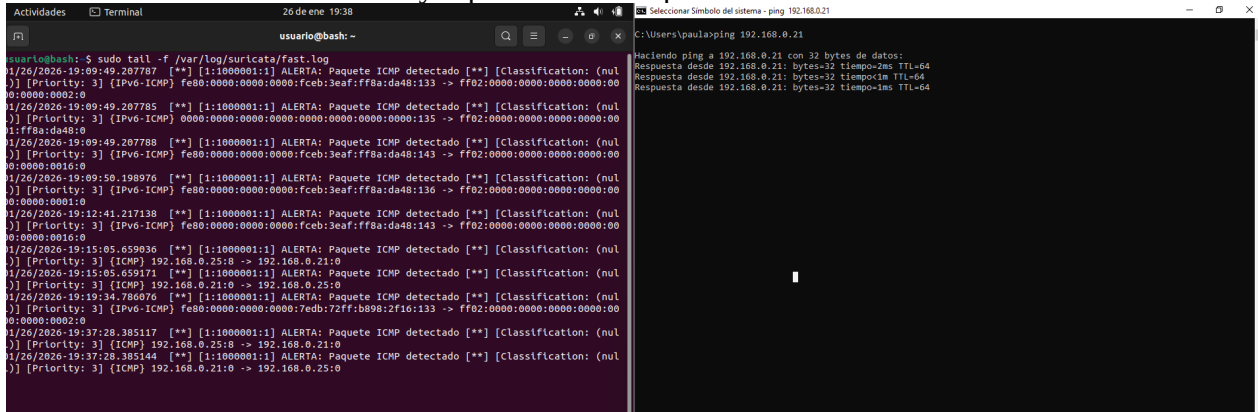


Ahora tenemos que modificar la variable HOME_NET para que sea más eficiente el IDS.



Repetir comprobación final

Volvemos a reiniciar el servicio y repetimos la comprobación de antes.



The screenshot shows a terminal window with two panes. The left pane displays a log file with several entries indicating detected ICMP packets. The right pane shows the output of a ping command to 192.168.0.21.

```
usuario@bash: ~  
usuario@bash:~$ sudo tail -f /var/log/suricata/fast.log  
01/26/2026-19:09:49.207787  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [IPv6-ICMP] fe80:0000:0000:0000:fceb:3eaf:ff8a:da48:133 -> ff02:0000:0000:0000:0000:00  
00:0000:0002:0  
01/26/2026-19:09:49.207785  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [IPv6-ICMP] 0000:0000:0000:0000:0000:0000:0000:0000:135 -> ff02:0000:0000:0000:0000:00  
1:ff8a:da48:0  
01/26/2026-19:09:49.207788  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [IPv6-ICMP] fe80:0000:0000:0000:fceb:3eaf:ff8a:da48:143 -> ff02:0000:0000:0000:0000:00  
00:0000:0002:0  
01/26/2026-19:09:50.198976  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [IPv6-ICMP] fe80:0000:0000:0000:fceb:3eaf:ff8a:da48:136 -> ff02:0000:0000:0000:0000:00  
00:0000:0001:0  
01/26/2026-19:12:41.217138  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [IPv6-ICMP] fe80:0000:0000:0000:fceb:3eaf:ff8a:da48:143 -> ff02:0000:0000:0000:0000:00  
00:0000:0016:0  
01/26/2026-19:15:05.659036  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [ICMP] 192.168.0.25:0 -> 192.168.0.21:0  
01/26/2026-19:15:05.659171  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [ICMP] 192.168.0.21:0 -> 192.168.0.25:0  
01/26/2026-19:19:34.786076  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [IPv6-ICMP] fe80:0000:0000:0000:7edb:72ff:b898:2f16:133 -> ff02:0000:0000:0000:0000:00  
00:0000:0002:0  
01/26/2026-19:37:28.385117  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [ICMP] 192.168.0.25:8 -> 192.168.0.21:0  
01/26/2026-19:37:28.385144  [**] [1:1000001:1] ALERTA: Paquete ICMP detectado [**] [Classification: (nul  
)] [Priority: 3] [ICMP] 192.168.0.21:0 -> 192.168.0.25:0
```

Seleccionar Símbolo del sistema - ping 192.168.0.21
C:\Users\paulazping 192.168.0.21
Haciendo ping a 192.168.0.21 con 32 bytes de datos:
Respuesta desde 192.168.0.21: bytes=32 tiempo=2ms TTL=64
Respuesta desde 192.168.0.21: bytes=32 tiempo<1ms TTL=64
Respuesta desde 192.168.0.21: bytes=32 tiempo<1ms TTL=64